

SystemVerilog RTL Bug Study

Geeth Sathsara Wijesekara

Introduction

This report examines five documented RTL design bugs from the open-source projects Corundum [10], OpenTitan [22], and CVA6 [40]. Each bug is reduced to a minimal reproducer and implemented in both SystemVerilog and Anvil [55], [56], then verified using standard industry approaches and the Anvil compiler where applicable. The report discusses how these bugs escaped their original verification environments and whether Anvil's compiler checks could have detected them earlier.

Bug1: Partial queue-memory initialization

1.1 Instance

This bug comes from Corundum [10], an open-source high-performance FPGA-based network interface card (NIC).

The three affected queue-management modules [11], [12], [13] store per-queue state in inferred FPGA RAM. During initialization, every RAM entry must be set to zero so that every legal queue index begins in an empty state.

In commit 0560f98e [16], a split nested loop was added to support more than 4,000 queues while working around Quartus's loop-iteration limit. Quartus Prime is Intel's FPGA toolchain [20]. This workaround introduced the bug discussed below; it was later corrected in PR #96 [17] and its fixing commit [18].

1.2 Mechanism

The above workaround confused the address width with the number of entries. The RAM contains $2 \times \text{QUEUE_INDEX_WIDTH}$ entries, but the new outer loop stopped at QUEUE_INDEX_WIDTH .

```
// Buggy code
for (i = 0; i < QUEUE_INDEX_WIDTH;
     i = i + 2**((QUEUE_INDEX_WIDTH/2)))
  for (j = i; j < i + 2**((QUEUE_INDEX_WIDTH/2)); j = j + 1)
    queue_ram[j] = 0;
```

Therefore, some entries remain uninitialized. The fix changes the outer bound to $2 \times \text{QUEUE_INDEX_WIDTH}$.

```
// Fixed code
for (i = 0; i < 2**QUEUE_INDEX_WIDTH;
     i = i + 2**((QUEUE_INDEX_WIDTH/2)))
  for (j = i; j < i + 2**((QUEUE_INDEX_WIDTH/2)); j = j + 1)
    queue_ram[j] = 0;
```

The minimal buggy reproducers in SystemVerilog [57] and Anvil [58], together with their fixed SystemVerilog [59] and Anvil [60] versions, are available in the cited reproduction files.

1.3 Why It Survived

At the pre-fix commit, Corundum's cocotb tests had no full-memory or X-state check. Both used this queue range [19], [14], [15]:

```
QUEUE_INDEX_WIDTH = 8

for q in range(4):
    configure_queue(q)

q = random.randrange(4)
exercise_queue(q)
```

An 8-bit index provides 256 entries, but the tests used only queues 0 through 3. The faulty loop initialized entries 0 through 15, so every tested queue appeared correct.

The loop was legal syntax, so lint could not infer the missing addresses. The fix added no full-range test or X-state assertion [17].

1.4 Classification

This bug can be classified as a **Reset and initialization** bug.

Rocket Chip provides a second independent instance of this class [21]. Its SRAMs were not initialized, which caused X propagation in simulation.

1.5 Detection

More generally, reset and initialization bugs can be detected by checking that all relevant state reaches a valid value after reset. Lint can flag state that is never reset, while simulation and formal properties can verify the expected post-reset values.

```
reset_complete |-> all_required_state_is_valid
```

For this reproducer, the concrete invariant is that every queue-memory entry is known and zero.

The simulation implementation [61]:

```
#1;
for (int q = 0; q < QUEUE_COUNT; q++)
  assert (!$isunknown(queue_ram[q * 32 +: 32]) &&
         queue_ram[q * 32 +: 32] == '0);
```

The corresponding formal property is implemented in [62]:

```
property p_entry_is_initialized(logic [31:0] entry);
  @(posedge f_clk)
  !$isunknown(entry) && entry == '0;
endproperty

for (genvar q = 0; q < QUEUE_COUNT; q++) begin
  assert property (
    p_entry_is_initialized(f_queue_ram[q * 32 +: 32]));
end
```

Simulation is cheap for this small memory, and its runtime grows linearly with the number of entries. Formal checking is exhaustive, but its state space grows with the memory. Lint may also report state that is deliberately left unreset, so its warnings need to be reviewed or waived.

Anvil’s checker generally cannot detect this type of bug, and it accepts both versions. Its generated RTL initializes the full register, masking this particular bug [58], [60], [63].

Bug2: Stale-data exposure from an empty FIFO

2.1 Instance

This bug comes from OpenTitan [22], an open-source silicon root-of-trust project. It is a bit different from the other RTL bugs in the report.

The FIFO buffers incoming UART words and exposes the previously received word when it is empty. Since this is visible to software, an empty FIFO must not reveal a byte previously received by another isolated application. The issue therefore identifies this as a **security** concern.

Issue #2315 [23] tracked the report. PR #2420 documents and corrects the stale-data exposure [24], with the change merged in commit [25].

2.2 Mechanism

The FIFO reset its pointers but not its storage. Its output remained connected to the stored word even when the FIFO was empty.

```
// Buggy code
if (Pass)
  assign rdata = (fifo_empty && wvalid) ? wdata : storage_rdata;
else
  assign rdata = storage_rdata;
```

Therefore, removing the last byte cleared the valid state but left that byte visible to the software. The fix masks the output with zero when the FIFO is empty.

```
// Fixed code
assign rdata = empty ? '0 : rdata_int;
```

The buggy and fixed SystemVerilog variants are provided separately [64], [65], together with their Anvil versions [66], [67].

2.3 Why It Survived

OpenTitan checked the FIFO output only while `rvalid` was high [26], [23], [24]:

```
assert property (rvalid |> !$isunknown(rdata));
```

When the FIFO became empty, this assertion stopped checking `rdata`. The UART tests also read data only from a non-empty FIFO [27], [28].

The failing operation was therefore not tested. The fix added no test or assertion [24].

2.4 Classification

This bug can be classified as a **Buffering, FIFOs, and flow control** bug.

The assignment already gives the OpenCores UART16550 as a second instance. It used stale receive-FIFO data without first checking whether the FIFO was empty [49].

2.5 Detection

More generally, buffering and flow-control bugs can be detected by checking that interface outputs and transfer decisions agree with the buffer’s occupancy state. Simulation and formal properties can check these relationships on every cycle.

```
transfer |> valid && ready && occupancy_allows_transfer
```

For this reproducer, the concrete invariant is that the externally visible FIFO output is zero whenever the FIFO is empty.

The simulation implementation [68]:

```
assert (empty && rdata == '0)
else $fatal(1, "empty FIFO exposed %h", rdata);
```

The corresponding formal property is implemented in [69]:

```
always_ff @(posedge f_clk) begin
  if (!f_rst_n)
    f_past_valid <= 1'b0;
  else
    f_past_valid <= 1'b1;
end

property p_reset_before_past_valid;
  @(posedge f_clk)
  !f_past_valid |> !f_rst_n;
endproperty: p_reset_before_past_valid

property p_empty_data_is_zero;
  @(posedge f_clk) disable iff (!f_rst_n)
  f_past_valid && f_empty |> (f_rdata == '0);
endproperty: p_empty_data_is_zero

assume property (p_reset_before_past_valid);

a_empty_data_is_zero:
  assert property (p_empty_data_is_zero);
```

The check is cheap, but the fix also adds a data-width multiplexer.

Anvil’s checker generally cannot detect this type of bug because the retained storage is not modified while borrowed. It accepts both versions, so the invariant must be checked separately [66], [67], [70].

Bug3: Unqualified asynchronous-FIFO payload crossing

3.1 Instance

This bug is from OpenTitan [22], in the asynchronous FIFO used by its TileLink clock-domain-crossing (CDC) path.

The FIFO carries request and response payloads between the host and device clocks. Its synchronized control shall

ensure that destination-domain logic observes the payload only after the source write is stable.

Issue #13834 [29] documented the unqualified asynchronous path. The behavior was corrected in PR #13991 [30] and its fixing commit [31].

3.2 Mechanism

The asynchronous FIFO synchronized its valid indication into the read clock domain, but exposed the associated multi-bit payload without gating it with the synchronized valid signal.

```
// Buggy code
assign rdata_o = rdata_int;
```

Hence, a source-domain write may toggle the destination-visible payload during the synchronization delay. The fix drives the output net to zero while the synchronized valid is low.

```
// Fixed code
assign rdata_o = rvalid_o ? rdata_int : '0;
```

The buggy and fixed SystemVerilog variants are provided separately [71], [72], together with their Anvil versions [73], [74].

To keep the SystemVerilog reproducer minimal, both variants ignore writes after the first. The Anvil versions retain repeated transfers because Anvil's channel semantics express and check payload lifetimes with less code.

3.3 Why It Survived

The pre-fix TL-UL testbench had no property for payload activity while `rvalid_o` was low [32], [33]. A CDC tool found this unchecked asynchronous path [29], [30].

RTL simulation cannot model analog metastability. The fix changed two RTL files and added no test or assertion [30].

3.4 Classification

This bug can be classified as a **Clock-domain crossing and metastability** bug.

The assignment gives the historical OpenCores UART16550 receive path as a second instance. It used the asynchronous receive input directly before commit 780ac1c added synchronizer flops [50].

3.5 Detection

More generally, CDC bugs can be detected with structural analysis that identifies unsynchronized paths and unsafe multi-bit crossings. Simulation and formal properties can complement this analysis by checking how the destination uses the crossing.

```
reject unsynchronized control and unqualified payload crossings
```

For this reproducer, the concrete invariant is that the destination payload remains zero until its synchronized valid signal is asserted.

The simulation implementation [76]:

```
assert (rvalid || rdata == '0)
else $fatal(1, "payload changed before valid");
```

The corresponding formal property is implemented in [77]:

```
always_ff @(posedge f_rd_clk) begin
  if (!f_rst_n)
    f_past_valid <= 1'b0;
  else
    f_past_valid <= 1'b1;
end

property p_reset_before_past_valid;
  @(posedge f_rd_clk) disable iff (!f_rst_n)
    !f_past_valid |-> !f_rst_n;
endproperty: p_reset_before_past_valid

property p_invalid_payload_is_zero;
  @(posedge f_rd_clk) disable iff (!f_rst_n)
    f_past_valid && !f_rvalid |-> (f_rdata == '0);
endproperty: p_invalid_payload_is_zero

assume property (p_reset_before_past_valid);

a_invalid_payload_is_zero:
  assert property (p_invalid_payload_is_zero);
```

CDC analysis requires correct clock constraints and reviewed waivers. The property is cheap but cannot model metastability, and the fix adds a multiplexer.

Anvil rejects the buggy channel model because it overwrites a register while an outstanding send borrows it. The fixed model keeps the payload stable and is accepted [73], [74], [78].

Bug4: Delayed backpressure across a four-phase handshake

4.1 Instance

This bug also comes from OpenTitan [22], in the entropy conditioner and its SHA3/AES power-management handshake.

The conditioner collects a fixed message and requests that AES halt before SHA3 processing to avoid a simultaneous power spike. Once this request begins, upstream entropy must be backpressured until processing completes and the four-phase request/acknowledge exchange returns to idle.

The handshaking delay was corrected in PR #15854 [34] and its implementation commit [35].

4.2 Mechanism

The conditioner used a registered ready mask to avoid a combinational loop, but the mask remained open until the delayed SHA3 processing event.

```
// Buggy code
assign sha3_msg_end = sha3_process;
mask_d = sha3_start ? 1'b1 : sha3_msg_end ? 1'b0 : mask_q;
```

Therefore, additional words could be accepted while the design waited for the previous AES acknowledgement to return low. The fix clears the stored ready mask when `cs_aes_halt_req` is asserted. It also waits for the

acknowledgement to return low before allowing the next operation.

```
// Fixed code
assign sha3_msg_end = cs_aes_halt_req;
mask_d = sha3_start ? 1'b1 : sha3_msg_end ? 1'b0 : mask_q;
```

The buggy and fixed SystemVerilog variants are provided separately [80], [81], together with their Anvil versions [82], [83].

4.3 Why It Survived

The UVM environment randomized AES handshake delays, but did not check that message acceptance remained blocked until the four-phase acknowledgement returned low [36], [37], [38], [39].

The fix changed two RTL files and added no test or assertion [35].

4.4 Classification

This bug can be classified as a **Handshake and channel protocol** bug.

The assignment already gives the Xilinx AXI-Lite demo slave as a second instance. It accepted a transfer using conditions beyond the interface's VALID-and-READY handshake [54].

4.5 Detection

More generally, handshake bugs can be detected by checking when transfers are accepted and by tracking each request through the required protocol sequence. Simulation and formal properties can check both safety and eventual completion.

```
accepted |-> valid && ready
request_started |-> required_sequence_holds_until_idle
```

For this reproducer, the concrete invariant is that readiness remains low from the time the request is acknowledged until ACK returns low.

The simulation implementation [85]:

```
assert (accepted == 2 && !msg_ready)
else $fatal(1, "started the next message before ack returned low");
```

The corresponding formal property is implemented in [86]:

```
always_ff @(posedge f_clk) begin
  if (!f_rst_n) begin
    f_past_valid <= 1'b0;
    f_ack_pending <= 1'b0;
  end else begin
    f_past_valid <= 1'b1;
    if (!f_aes_ack)
      f_ack_pending <= 1'b0;
    else if (f_aes_req)
      f_ack_pending <= 1'b1;
  end
end

property p_reset_before_past_valid;
  @(posedge f_clk)
  !f_past_valid |-> !f_rst_n;
endproperty: p_reset_before_past_valid
```

```
property p_reset_remains_deasserted;
  @(posedge f_clk)
  f_past_valid |-> f_rst_n;
endproperty: p_reset_remains_deasserted

property p_ack_pending_blocks_input;
  @(posedge f_clk) disable iff (!f_rst_n)
  f_ack_pending |-> !f_msg_ready;
endproperty: p_ack_pending_blocks_input

property p_ack_eventually_asserts;
  @(posedge f_clk) disable iff (!f_rst_n)
  f_past_valid && f_aes_req |->
    s_eventually f_aes_ack;
endproperty: p_ack_eventually_asserts

property p_ack_eventually_deasserts;
  @(posedge f_clk) disable iff (!f_rst_n)
  f_past_valid && f_aes_ack |->
    s_eventually !f_aes_ack;
endproperty: p_ack_eventually_deasserts

property p_handshake_eventually_returns_idle;
  @(posedge f_clk) disable iff (!f_rst_n)
  f_past_valid && f_aes_req |->
    s_eventually (!f_aes_req && !f_aes_ack);
endproperty: p_handshake_eventually_returns_idle

assume property (p_reset_before_past_valid);
assume property (p_reset_remains_deasserted);
assume property (p_ack_eventually_asserts);
assume property (p_ack_eventually_deasserts);

assert property (p_ack_pending_blocks_input);
assert property (p_handshake_eventually_returns_idle);
```

Unlike the other formal checks in this report, the handshake-completion check is a liveness property. It requires fairness assumptions for the environment: ACK must eventually rise after a request and eventually fall. Without these assumptions, the tool can produce a counterexample in which the environment holds ACK forever. The formal setup is available here [87].

Liveness checks can be more expensive than safety checks. Insufficient fairness assumptions may prevent a proof. Assumptions that are too strong or unrealistic may make the result meaningless.

Anvil rejects the buggy channel model because it overwrites a borrowed payload while the acknowledgement is outstanding. The fixed blocking-handshake model is accepted [82], [83], [88].

Bug5: CSR instruction incorrectly allowed to dual-issue

5.1 Instance

This bug comes from CVA6 [40], an open-source RISC-V processor, in its superscalar issue stage.

The issue stage can dispatch two ordinary instructions if they are proven to be independent. However, CSR operations access privileged state and can redirect, trap, or change the context, so they must be serialized.

The issue restriction was corrected in PR #2329 [41], through the CSR-hazard commit [42] and CSR-port commit [43].

5.2 Mechanism

The issue-stage exclusion vector blocked only selected functional units when slot 0 contained a CSR instruction.

```
// Buggy code
CSR: begin
  fus_busy[1].alu      = 1'b1;
  fus_busy[1].ctrl_flow = 1'b1;
  fus_busy[1].csr      = 1'b1;
end
```

Therefore, other instruction classes could issue with a CSR, and a CSR could also reach the second lane. The fix blocks every functional unit from the second lane when slot 0 contains a CSR and ensures that a CSR uses only lane 0.

```
// Fixed code
fus_busy[1].csr = 1'b1;

CSR: begin
  fus_busy[1] = '1;
end
```

The buggy and fixed SystemVerilog variants are provided separately [89], [90], together with their Anvil versions [91], [92].

5.3 Why It Survived

The PR does not state how the bug was found. A visible failure requires a CSR to issue beside another instruction and affect its behavior.

CVA6 had CSR tests and an RVFI/UVM scoreboard [47], [48]. However, its default CI target disabled superscalar issue and could not exercise the bug [44], [45], [46].

Both fixes changed only `issue_read_operands.sv` and added no test or assertion [42], [43].

5.4 Classification

This bug can be classified as a **Pipeline control and hazards** bug.

XiangShan provides a second instance of this class. Its rename-stage bypass used the wrong dependency tag, so `csrr vl` immediately following `vsetvli` read stale state [51], [52].

5.5 Detection

More generally, pipeline-control and hazard bugs can be detected by checking each instruction selected for issue against the design's dependency and serialization rules. Simulation and formal properties can exercise different combinations of instruction classes.

```
issued(i) |-> dependencies_clear(i) && serialization_allows(i)
```

For this reproducer, the concrete invariant is that a CSR instruction never issues beside another instruction and never uses the second issue lane.

The simulation implementation [93]:

```
assert (!issue1)
  else $fatal(1, "instruction issued beside CSR");
```

The corresponding formal properties are implemented in [94]:

```
property p_csr_is_serialized;
  @(posedge f_clk)
    f_issue0 && f_fu0 == CSR |-> !f_issue1;
endproperty: p_csr_is_serialized

property p_csr_uses_port_zero;
  @(posedge f_clk)
    !(f_issue1 && f_fu1 == CSR);
endproperty: p_csr_uses_port_zero
```

This pessimistic serialization avoids the bug, but it may be too restrictive and limit performance. However, CSRs are generally rare and uncommon in hot loops. The check itself is inexpensive.

Anvil's checker generally cannot detect this type of bug because it requires high-level system reasoning. It accepts both versions, so CSR serialization must be checked separately [91], [92], [95].

6. Synthesis

Bug	Class	Comments	Best technique	Anvil ability
1	Reset and initialization	The loop initializes only a part of a parameterized memory.	Four-state simulation or a formal completeness property over every entry.	Not detected. The generated RTL masks the difference.
2	Buffering, FIFOs, and flow control	An empty software-visible FIFO exposes its retained payload.	Empty=>zero assertion.	Not detected.
3	Clock-domain crossing and metastability	An asynchronous payload is visible before its synchronized valid signal.	Structural CDC analysis plus an invalid=>zero assertion.	Detected when modeled as a channel-lifetime violation.
4	Handshake and channel protocol	Delayed backpressure accepts extra input while acknowledgement is pending.	Protocol assertions with continuous input and delayed ACK.	Detected when modeled as a channel-lifetime violation.
5	Pipeline control and hazards	A CSR can issue beside another instruction or on the wrong port.	Issue serialization assertions and formal instruction pairing.	Not detected.

6.1 Anvil Modeling Requirement

For Bugs 3 and 4, Anvil rejects the buggy versions when the designs are correctly expressed using its channel model [73], [82]. The corrected versions pass the compiler [74], [83]. However, code that models the channel incorrectly can still pass Anvil’s checks while remaining functionally wrong [75], [84].

6.2 Simulation

The simulations use the open-source SystemVerilog/Verilog simulator Verilator 5.051, revision 7d3021c34 [7].

Verilator is a two-state simulator. It is also cycle-based, although that is not relevant here. Hence, uninitialized values are not preserved as **X**, so both Bug 1 versions pass.

6.3 Model Checking

The formal checks use the open-source SymbiYosys tool, version 0.67, revision fea6e46 [8]. BMC finds counterexamples in the buggy versions. The fixed versions use *k*-induction for Bug 1 and PDR for Bugs 2–5. Bug 4 also checks liveness with *suprove*.

6.4 CDC

For Bug 3, rtl-buddy-cdc reported the buggy payload as an unprotected bus crossing. It did not report this issue for the fixed, valid-gated payload [9], [79].

6.5 Selected Projects

The selected projects are Corundum [10], OpenTitan [22], and CVA6 [40].

- OpenTitan includes a UVM and formal-verification suite, and the selected bugs have clear issue-PR-commit links.
- CVA6 includes processor, ISA, and UVM verification infrastructure.
- Corundum uses cocotb/pytest regressions and does not include UVM or formal checks.

6.6 Test Matrix

Bug	Sim/SV buggy [1]	Sim/SV fixed [1]	Compile						Sim/AnvilSim/AnvilSim/Anvil						Formal	
			Formal/SV buggy [2]	Formal/SV fixed [2]	SV buggy [3]	SV fixed [3]	Compile semantic [4]	Compile fixed [3]	Sim/Anvil buggy [5]	Sim/Anvil semantic [5]	Sim/Anvil fixed [5]	Formal buggy [6]	Formal semantic [6]	Formal fixed [6]		
1	PASS	PASS	Safety; BMC; Z3; d2; FAIL/SAT	Safety; BMC+k- ind.; Z3; d2; PASS	PASS	PASS	—	PASS	PASS (masked)	—	PASS	Safety; BMC; Z3; d6; PASS (masked)	—	Safety; BMC+k- ind.; Z3; d6; PASS		
2	FAIL	PASS	Safety; BMC; Z3; d8; FAIL/SAT	Safety; PDR; ABC; PASS	PASS	PASS	—	PASS	FAIL	—	PASS	Safety; BMC; Z3; d12; FAIL/SAT	—	Safety; PDR; ABC; PASS		
3	FAIL	PASS	Safety; BMC; Z3; d8; FAIL/SAT	Safety; PDR; ABC; PASS	REJECT	PASS	PASS	PASS	Not run	FAIL	PASS	Not run	Safety; BMC; Z3; d12; FAIL/SAT	Safety; PDR; ABC; PASS		
4-S	FAIL	PASS	Safety; BMC; Z3; d10; FAIL/SAT	Safety; PDR; ABC; PASS	REJECT	PASS	PASS	PASS	Not run	FAIL	PASS	Not run	Safety; BMC; Z3; d16; FAIL/SAT	Safety; PDR; ABC; PASS		
4-L	—	—	Live; suprove; PASS	Live; suprove; PASS	—	—	—	—	—	—	—	—	—	Live; suprove; PASS		
5	FAIL	PASS	Safety; BMC; Z3; d2; FAIL/SAT	Safety; PDR; ABC; PASS	PASS	PASS	—	PASS	FAIL	—	PASS	Safety; BMC; Z3; d12; FAIL/SAT	—	Safety; PDR; ABC; PASS		

References

- [1] SystemVerilog simulation environments: `src/bug1`, `src/bug2`, `src/bug3`, `src/bug4`, `src/bug5`.
- [2] SystemVerilog formal environments: `src/bug1/formal`, `src/bug2/formal`, `src/bug3/formal`, `src/bug4/formal`, `src/bug5/formal`.
- [3] Buggy and fixed Anvil sources: `src/bug1/rtl`, `src/bug2/rtl`, `src/bug3/rtl`, `src/bug4/rtl`, `src/bug5/rtl`.
- [4] Anvil semantic-bug sources: `src/bug3/rtl/semantic_bug.anvil`, `src/bug4/rtl/semantic_bug.anvil`.
- [5] Generated-Anvil simulation testbenches: `src/bug1/tb/anvil_tb.sv`, `src/bug2/tb/anvil_tb.sv`, `src/bug3/tb/anvil_tb.sv`, `src/bug4/tb/anvil_tb.sv`, and `src/bug5/tb/anvil_tb.sv`.
- [6] Generated-Anvil formal configurations, properties, and logs: `src/bug1/formal`, `src/bug2/formal`, `src/bug3/formal`, `src/bug4/formal`, `src/bug5/formal`.
- [7] Verilator revision 7d3021c34d0f26a88e8b1107c50ceb2d896689b3.
- [8] SymbiYosys version 0.67, revision fea6e467d067b3ea84b6b5ac08cd48beb59f0d42.
- [9] rtl-buddy-cdc version 0.5.0, revision df996d55af0e8b824183d03d531426b8601af429.
- [10] Corundum source repository.
- [11] Corundum `queue_manager` source.
- [12] Corundum `cpl_queue_manager` source.
- [13] Corundum `tx_scheduler_rr` source.
- [14] Corundum pre-fix `queue_manager` regression test.
- [15] Corundum pre-fix `cpl_queue_manager` regression test.
- [16] Corundum: Support more than 4k queues.
- [17] Corundum PR #96: Correct queue-memory initialization.
- [18] Corundum fixing commit 7bc1ecc.
- [19] Corundum pre-fix parent commit 0f82e0c.
- [20] Intel Quartus Prime FPGA design software.
- [21] Rocket Chip: Initialize all SRAMs to avoid X propagation.
- [22] OpenTitan source repository.
- [23] OpenTitan issue #2315: Stale data from empty UART RX FIFO.
- [24] OpenTitan PR #2420: Fix empty-FIFO stale data.
- [25] OpenTitan merged commit 9ac2b15.
- [26] OpenTitan stale-FIFO pre-fix parent commit 0408a4e.
- [27] OpenTitan pre-fix UART test plan.
- [28] OpenTitan pre-fix UART scoreboard.
- [29] OpenTitan issue #13834: W_MASYNC in `spi_device`.
- [30] OpenTitan PR #13991: Qualify asynchronous-FIFO output.
- [31] OpenTitan CDC-fix commit e0ba6cd.
- [32] OpenTitan CDC pre-fix parent commit 592eaf6.
- [33] OpenTitan pre-fix TL-UL verification environment.
- [34] OpenTitan PR #15854: Fix handshaking delay.
- [35] OpenTitan handshake-fix commit cd1b613.
- [36] OpenTitan handshake pre-fix parent commit 78944e3.
- [37] OpenTitan pre-fix entropy-source AES handshake agent.
- [38] OpenTitan pre-fix randomized handshake-delay configuration.
- [39] OpenTitan pre-fix entropy-source scoreboard.
- [40] CVA6 source repository.
- [41] CVA6 PR #2329: Do not issue CSR with another instruction.
- [42] CVA6 CSR-hazard commit 92e418a.
- [43] CVA6 CSR-port commit 827daa0.
- [44] CVA6 pre-fix parent commit 67dba2c.
- [45] CVA6 pre-fix CI configuration.
- [46] CVA6 pre-fix cv32a65x configuration.
- [47] CVA6 pre-fix CSR regression.
- [48] CVA6 pre-fix RVFI/UVFV scoreboard.
- [49] OpenCores UART16550 receive-FIFO RTL.
- [50] OpenCores UART16550 commit 780ac1c: add receive-input synchronizer flops.
- [51] XiangShan issue #5739: `csrr vl` reads stale zero immediately after `vsetvli`.
- [52] XiangShan PR #5743: fix vector-length dependency bypass in rename.
- [53] Intel processor specification update: SKL150/KBL095.
- [54] D. Gisselquist, "Examining Xilinx's AXI demonstration core." J. Z. Yu, A. R. Jha, U. Mathur, T. E. Carlson, and P. Saxena, "Anvil: A General-Purpose Timing-Safe Hardware Description Language," ASPLOS, 2026.
- [55] Anvil compiler source repository.
- [56] `src/bug1/rtl/buggy.sv`.
- [57] `src/bug1/rtl/buggy.anvil`.
- [58] `src/bug1/rtl/fixed.sv`.
- [59] `src/bug1/rtl/fixed.anvil`.
- [60] `src/bug1/tb/tb.sv`.
- [61] `src/bug1/formal/properties.sv`.
- [62] `src/bug1/formal/results/logs/anvil_compile_buggy.log`;
- [63] `src/bug1/formal/results/logs/anvil_compile_fixed.log`;
- `src/bug1/formal/results/logs/anvil_buggy.log`; `src/bug1/formal/results/logs/anvil_fixed.log`.
- [64] `src/bug2/rtl/buggy.sv`.
- [65] `src/bug2/rtl/fixed.sv`.
- [66] `src/bug2/rtl/buggy.anvil`.
- [67] `src/bug2/rtl/fixed.anvil`.
- [68] `src/bug2/tb/tb.sv`.
- [69] `src/bug2/formal/properties.sv`.
- [70] `src/bug2/formal/results/logs/anvil_compile_buggy.log`;
- `src/bug2/formal/results/logs/anvil_compile_fixed.log`;
- `src/bug2/formal/results/logs/anvil_buggy.log`; `src/bug2/formal/results/logs/anvil_fixed.log`.
- [71] `src/bug3/rtl/buggy.sv`.
- [72] `src/bug3/rtl/fixed.sv`.
- [73] `src/bug3/rtl/buggy.anvil`.
- [74] `src/bug3/rtl/fixed.anvil`.
- [75] `src/bug3/rtl/semantic_bug.anvil`.
- [76] `src/bug3/tb/tb.sv`.
- [77] `src/bug3/formal/properties.sv`.
- [78] `src/bug3/formal/results/logs/anvil_compile_buggy.log`;
- `src/bug3/formal/results/logs/anvil_compile_semantic_bug.log`;
- `src/bug3/formal/results/logs/anvil_compile_fixed.log`;
- `src/bug3/formal/results/logs/anvil_semantic_bug.log`;
- `src/bug3/formal/results/logs/anvil_fixed.log`.
- [79] `src/bug3/cdc/results/buggy.log`; `src/bug3/cdc/results/fixed.log`.
- [80] `src/bug4/rtl/buggy.sv`.
- [81] `src/bug4/rtl/fixed.sv`.
- [82] `src/bug4/rtl/buggy.anvil`.
- [83] `src/bug4/rtl/fixed.anvil`.
- [84] `src/bug4/rtl/semantic_bug.anvil`.
- [85] `src/bug4/tb/tb.sv`.
- [86] `src/bug4/formal/properties.sv`.
- [87] `src/bug4/formal/sby_properties.sv`; `src/bug4/formal.sby`;
- `src/bug4/formal.live.sby`.
- [88] `src/bug4/formal/results/logs/anvil_compile_buggy.log`;
- `src/bug4/formal/results/logs/anvil_compile_semantic_bug.log`;
- `src/bug4/formal/results/logs/anvil_compile_fixed.log`;
- `src/bug4/formal/results/logs/anvil_semantic_bug.log`;
- `src/bug4/formal/results/logs/anvil_fixed.log`.
- [89] `src/bug5/rtl/buggy.sv`.
- [90] `src/bug5/rtl/fixed.sv`.
- [91] `src/bug5/rtl/buggy.anvil`.
- [92] `src/bug5/rtl/fixed.anvil`.
- [93] `src/bug5/tb/tb.sv`.
- [94] `src/bug5/formal/properties.sv`.
- [95] `src/bug5/formal/results/logs/anvil_compile_buggy.log`;
- `src/bug5/formal/results/logs/anvil_compile_fixed.log`;
- `src/bug5/formal/results/logs/anvil_buggy.log`; `src/bug5/formal/results/logs/anvil_fixed.log`.
- [96] Source inventory: `docs/sources.txt`.

Revision History

- 1: Replaced the second example for Bug 5, Intel's partial-register erratum [53], with the more relevant XiangShan dependency bug [51], [52].
- 2: Generalized the detection discussions for Bugs 1-5.
- 3: Clarified Bug 3's reduction scope.
- 4: Fix the Bug 4 reproducer.